mininet: A Shared-Nothing HTTPS Stack for Cloud-Native Analytical Engines

Martin Lindbüchl
martin.lindbuechl@tum.de
Technical University of Munich
Munich, Germany

Abstract

Cloud-native analytical engines increasingly leverage object stores for persistent data. This design of disaggregated compute and storage provides elasticity and high data durability. However, it requires moving large amounts of data between physically separated machines, making the network stack a performance-critical component of modern OLAP engines. This motivates the question of whether to rely on kernel networking or opt for a kernel-bypass solution. We argue that neither is a good fit. While kernel networking provides good bandwidth for bulk transfers, it incurs high CPU overhead. Kernel bypass stacks, on the other hand, are more complex to integrate with existing databases and often optimize for latency rather than bandwidth.

This paper presents mininet, a kernel-native, shared-nothing HTTP(S) stack, tailored to the workload of analytical engines. Our design is easy to integrate into existing analytical engines while providing higher bandwidth on fewer CPU cores than the Linux network stack. Our prototype reaches over 70 Gbit/s on a 2-core budget and over 160 Gbit/s on an 8-core budget, and our 400 LoC integration of mininet into DuckDB speeds up the TPC-H benchmark by 1.22× on average over all queries.

1 Introduction

The demand for elasticity in the cloud is shifting the landscape of analytical engines towards a design that disaggregates storage from compute resources [1, 8, 30]. The resulting engines leverage object stores for persistent data, thereby shifting from fetching cold data from local NVMe SSDs to fetching it over the network. This places the network stack on the critical path [10], raising the question of whether the kernel network stack or a kernel-bypass stack is a better fit for the high-bandwidth workload of analytical query processing.

We argue that neither stack fulfills all requirements set by the database engine. Although the kernel stack in Linux excels at bulk transfer, as filling a link with large packets hides the software overhead of preparing the next segment [2], the complexity and system call overhead incur a high CPU usage [10]. As the DBMS and the network stack compete for resources, this diverts CPU cores from the database engine into the network stack, causing performance penalties. Kernel bypass stacks can significantly reduce CPU usage, but they are difficult to integrate into existing database engines [32]. Additionally, many kernel-bypass stacks that include a TCP implementation optimize for latency or connection scalability, ultimately delivering worse bulk-transfer performance than the Linux kernel stack [2].

This paper proposes leveraging the unikernel architecture to make improving the operating system (OS) a feasible alternative to bypassing and re-implementing kernel functionality in user space. Unikernels further enable moving beyond dated POSIX APIs, such as sockets, and allow us to re-imagine the interfaces between the application and the network stack. As a concrete design implementing this vision, we present mininet, a standalone, shared-nothing HTTP(S) client networking stack. Mininet maintains multiple independent slices of its network stack, each operated by a single designated CPU core and responsible for a single network interface card (NIC) queue. Our design leverages predictable receive-side scaling (RSS) to ensure responding packets are steered to the networking slice that requested it, alleviating the need for any shared state or synchronization between networking cores. We implement mininet in the miniOSv unikernel [21] and integrate our prototype with DuckDB, adding only ~400 lines of code (LoC) to its codebase.

The evaluation of our prototype on AWS VMs shows that we achieve 70 Gbit/s on a 2-core budget, 1.7× higher than the Linux networking stack. Our prototype is competitive with Linux and Anyblob with regard to maximum bandwidth at ~160 Gbit/s on our testbed. We further show that our prototype improves the geometric mean over all TPC-H queries in DuckDB by 1.22×.

2 Background

In this section, we examine the characteristics of cloud networking and discuss the Linux kernel and kernel-bypass stacks. Based on this discussion, we motivate the need for unikernel networking and highlight its current status in terms of functionality and performance.

2.1 Cloud Networking

Hyperscalers now offer networking bandwidth up to 600 Gbit/s [4], enabling fast transfers between object stores and EC2 instances. These transfers are usually HTTPS GET requests, therefore requiring a full TCP network stack. Note that for AWS, plain HTTP can also be used, significantly reducing CPU overhead. This can be done safely because (1) within one AZ, VPC isolation protects data from being intercepted by other users [10], and (2) across AZs or regions, AWS encrypts traffic at the physical layer [3].

Like most modern NICs, the AWS Elastic Network Adapter (ENA), used on EC2 instances, is capable of RSS. This means it exposes multiple receive queues and distributes incoming packets among them [12]. To this end, the NIC uses the hash of the source IP, source port, destination IP, and destination port to determine which queue the packet should be steered to. This allows multiple cores to work on the RX path without requiring synchronization. Furthermore, since the queue a flow will land on is predictable, it can be leveraged to steer a connection to a specific queue by choosing its source port accordingly.

2.2 Network Stacks

Efficiently leveraging object stores requires a fast, HTTP network stack with low resource usage. Linux is optimized for bulk transfers, well-maintained, and stable. It relies on the known POSIX interface, which most DBMS network layers already use, minimizing the engineering effort. Using the `io_uring` system call interface further reduces the cost of system calls. This exact design is utilized in the object store download manager anyblob [10]. The fundamental problem with the Linux stack is its complexity and the connected software overhead. As a general-purpose stack, Linux must support a wide range of workloads and TCP features. This caps the per-core maximum bandwidth to an unacceptable level for this workload, with anyblob reporting over 10 utilized CPUs for 73 Gbit/s plain HTTP download [10].

Kernel bypass stacks, on the other hand, often implement less TCP functionality and focus on a specific workload rather than generality. This allows fine-tuning the stack for the specific workload, thereby outperforming the Linux stack several-fold for small messages [2]. However, recent research in this field has focused on optimizing for latency and throughput, rather than bandwidth [5, 14, 15, 28, 31]. As a consequence, the Linux kernel stack still outperforms all of the above for bulk transfers [2]. In addition, kernel bypass stacks require significant engineering effort to integrate, are often poorly maintained, or require the application to pre-segment data into packet-sized chunks [2], making them unfit for the workload imposed by analytical engines.

Considering the benefits and drawbacks of both stacks, we argue that neither is a good fit for our workload. The Linux kernel stack incurs high CPU overhead, and kernel bypass stacks are not optimized for bandwidth and significantly harder to implement.

2.3 Unikernel Networking

The unikernel architecture offers several benefits for our workload: (1) By statically linking the application into the OS, they eliminate the overhead of system calls. (2) The resulting unikernel application is smaller, simpler, and more predictable than multi-process OSes. (3) The simplicity of the unikernel allows weaving a purpose-built network stack into the kernel, instead of going the more complex kernel bypass route. (4) The single-address-space design enables exposing APIs at different levels, thereby letting the application choose the degree of co-design with the kernel.

Existing unikernels that provide a network stack can be classified in 3 categories: (1) The ones that reuse the stack of a general-purpose OS [16, 18, 24]. (2) The ones that port a standalone network stack [17, 19]. (3) The ones that implement a network stack from scratch [7, 20]. Category 1 is undesirable for reasons already laid out above, leaving categories 2 and 3 feasible for our workload. Closer inspection of the stacks in these categories, however, shows that they are either confined to a single core [6, 7, 13, 17, 20] or, as in the case of Hermit, reach only a small fraction of a 100 Gbit/s link [11], making it impossible to saturate a multi-hundred Gbit/s link.

Given the constraints of all three unikernel network stack categories, we argue that there is currently no stack capable of saturating multi-hundred-gigabit links with only a few CPU cores involved in networking.

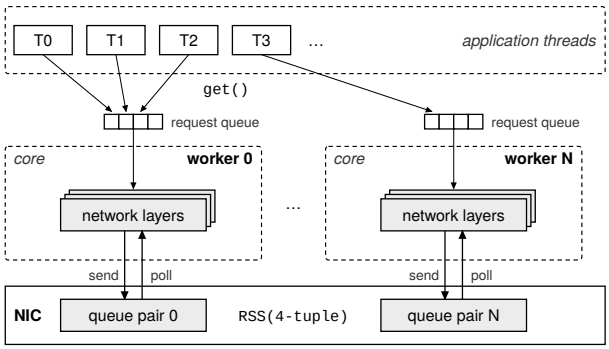


Figure 1: Mininet architecture overview: Designated networking threads on reserved cores own a vertical slice of the entire networking stack, from the TLS session down to one NIC queue pair.

3 Design

Our design vertically splits the network stack into independent slices, each responsible for a designated NIC queue. As shown in Figure 1, each vertical slice is assigned to an exclusive CPU core, reserved for networking. This section details how our design achieves concurrent operation on the same NIC without shared state or synchronization. We iterate over supported TCP/IP implementations and execution models and introduce the potential for co-design between application, network stack, and other kernel functionality.

3.1 Shared-nothing

The general idea of our design is that nothing on the network stack is shared between threads, allowing concurrent operation without synchronization overhead. This idea relies heavily on RSS to deterministically steer packets to a specific queue. The technique is known as *predictable RSS*, and already utilized by Data Plane Development Kit (DPDK) and mTCP [9, 14]. Predictable RSS builds on the fact that all information used to steer a packet to a specific queue is available before the message requesting this packet is sent. The hash parameters and the hash function are known, allowing the hash result to be computed in advance. The indirection table that maps the hash result to a specific queue is also known from boot. Consequently, by modifying the hash function parameters, we can actively choose the hash result and, therefore, the queue in which the answering packets should arrive. While most of the hash function's parameters (source IP, destination IP, and destination port) are fixed, we can choose the source port freely. Meaning, by carefully choosing a source port so that RSS steers packets back to the queue owned by the network slice, each slice ensures independent operation on its rx/tx queue pair.

Although this limits the number of available source ports for 2^n networking slices to 2^{14-n} source ports per slice, we argue this is sufficient due to the low number of slices needed to saturate the link. Another consequence of this design is that a new set of source Ports to dial from must be calculated when any of the 3 fixed hash parameters change. However, considering the HTTP-only workload

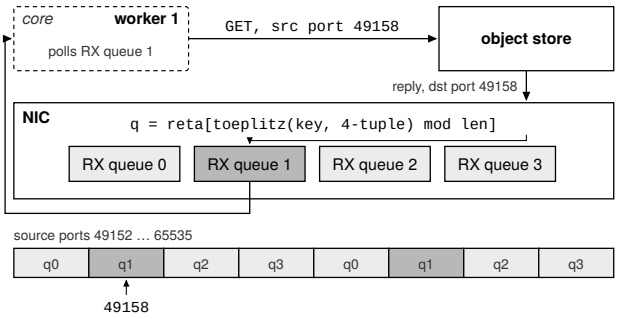


Figure 2: Predictable RSS: The RX queue a reply lands on is a static function with 4 parameters. By freely choosing one parameter - the source port - a worker can ensure the response will be steered to the queue it already polls.

from an EC2 instance to a long-living object store endpoint, this is a rare occurrence, producing negligible overhead.

Figure 2 exemplifies predictable RSS for a network slice responsible for queue pair 1 and source port 49158. This port is chosen in accordance with the other hash parameters, which we omit here, to produce a specific RSS hash associated with queue pair 1.

3.2 TCP/IP Implementations

Our design instantiates a complete TCP/IP stack on each networking slice, thereby removing the requirement of multicore scalability to the stack itself. In this environment, any single-core TCP/IP implementation can be used and scaled independently by leveraging predictive RSS. This design also simplifies extending the stack with additional functionality, including TLS.

3.3 Execution Model

We use a producer/consumer model for our high-level API. Figure 1 shows the corresponding request queues and `get()` API call. This model is beneficial for easily integrating analytical engines by providing a similar endpoint to the Linux kernel. This design reserves specific networking cores, thereby reducing noise from a competing query engine and network stack. It further improves instruction-cache locality on the networking cores and allows them to optimize the number of in-flight connections.

3.4 Application/Stack Co-design

Our design accounts for more sophisticated integration of the networking stack into the application by providing a run-to-completion model. This low-level API does not reserve CPU cores for networking. Instead, every application thread gets exclusive access to a NIC queue. HTTP requests in this model are served by the calling core through the entire network stack back to the application, maximizing data locality. Since this model only works for at most as many threads as there are NIC queues available, it requires specific handling in the application and complicates integration into existing engines.

4 Implementation

We implement our prototype on miniOSv [21], a radically slimmed-down version of OSv without POSIX support or existing networking capabilities. This section elaborates on the implementation of different layers of our mininet network stack.

Foundation. We ported the ENA NIC driver [29] and MiniDPDK [22] from OSv to miniOSv, ~9k and ~1k LoC respectively. MiniDPDK drives the NIC in poll mode and exposes its queues, RSS configuration, and per-frame metadata through a DPDK-like device abstraction, on which higher levels of the mininet stack build.

Mininet. Our prototype leverages our design's property to choose a single-core network stack while maintaining scalability. For Ethernet, IP, and TCP layers, we leverage the unmodified SmolTCP [27] Rust crate. SmolTCP is a standalone, device-agnostic library without its own runtime or event loop, making it an ideal fit for our design. We further leverage rusTLS [26] to implement TLS 1.3 [25]. This employed Rust crates handle protocol processing and TLS. We implement the worker model, predictable RSS, and connection management in Rust, adding another ~4k LoC. In total, the full path from NIC to database engine comprises ~14k LoC, of which ~10k are the ported driver and MiniDPDK layers and ~4k are our mininet code.

Analytical Engine. For the application side, we ported and integrated DuckDB [23] into our prototype. Of the ~4k LoC added to DuckDB, 3.6k LoC account for the port, while only 400 LoC where necessary to make DuckDB use the mininet networking stack. In this setup, we can leverage DuckDB's httpfs extension without modifications, as it allows selecting custom HTTP clients. Note that no optimizations have been implemented in DuckDB.

5 Evaluation

We evaluate the performance of mininet compared to the Linux kernel stack. First, we use a microbenchmark to investigate the reachable bandwidth relative to the number of CPU cores utilized. This comparison includes anyblob, a sophisticated download manager that leverages `io_uring`. We also evaluate the performance of the TPCH benchmark suite on DuckDB and investigate the origin of the performance benefits of mininet over Linux.

instance	vCPUs	memory	network
`r6in.8xlarge`	32	256 GiB	50 Gbit/s
`r6in.32xlarge`	128	1 TiB	200 Gbit/s

Table 1: The two instance types used in this evaluation.

Testbed. All experiments are run on network-optimized AWS virtual machines (VMs), with one ENA NIC to each instance. Table 1 lists the two instance types used in our evaluation.

Competitors. We compare our mininet prototype against the unmodified Linux kernel stack. For fairness, we disabled Generic Receive Offload (GRO) in Linux, as it is a software feature not implemented in our mininet prototype. Our bandwidth benchmark further includes an anyblob competitor, as it is designed specifically to reduce the CPU usage of the Linux kernel stack.

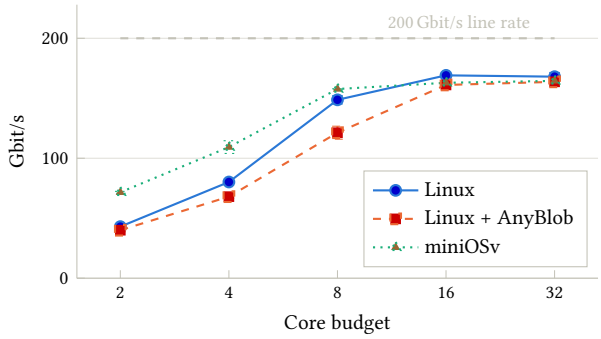


Figure 3: Average bandwidth of 32 MiB ranged HTTP GET requests on `r6in.32xlarge` per core budget.

5.1 Bulk Transfer Bandwidth

To measure the achievable bandwidth for bulk transfers, we set up an nginx server to process requests from our client network stacks. We use nginx rather than the object store here because AWS S3 endpoint performance varies significantly. In particular, the varying MTU sizes between endpoints [10] would influence the results of the experiment.

The results of miniosv, Linux, and Anyblob are shown in 3. For the particular workload, we see that miniosv can achieve a significantly higher bandwidth than its competitors, outperforming Linux by 1.7× and Anyblob by 1.8× on a 2-core budget. The experiment also shows that the miniosv stack converges around the same core count to a similar max bandwidth as its competitors. This upper limit is introduced by the testing infrastructure, rather than the client stack itself.

	miniOSv	Linux		AnyBlob	
budget	Gbit/s	Gbit/s	cores	Gbit/s	cores
2	71.6	43.0	2.00	40.0	1.58
4	109.3	80.2	4.00	68.0	2.39
8	157.7	148.7	7.97	121.3	3.36
16	162.9	169.1	13.46	161.1	4.46
32	164.5	168.0	16.27	163.7	5.62

Table 2: Measured core utilization. MiniOSv cores are always equal to the budget due to busy polling.

While the superiority of miniOSv over Linux is due to the network stack's simplicity and low-overhead design, it is unexpected that Linux without Anyblob performs better than the optimized object store download manager. This is mostly due to Anyblob not using its core budget. This is shown in Table 2. Since Linux dynamically schedules its networking threads, the core budget is only an upper limit; the Linux kernel and Anyblob do not have to utilize all of it. The lower core utilization in Anyblob explains why the standard Linux networking stack reaches a higher bandwidth in this experiment.

5.2 TPC-H Peformance

We use our naive DuckDB/miniOSv integration to measure end-to-end performance of our prototype compared to a traditional Linux VM. This benchmark uses the `r6in.8xlarge` AWS instance type. In our experiment, we leverage DuckDB's httpfs extension, which allows us to specify database tables as object store paths, where the compressed TPCH parquet files are stored. We use a scale factor of 100 for the TPC-H dataset. For both Linux and minioSv, we disable caching inside DuckDB to ensure all data is fetched over the network, rather than cached in main memory. We further noticed that oversubscribing the worker threads in DuckDB improved performance, as it increases the number of concurrent requests. Both competitors exhibit peak performance at 256 threads on the testbed, which corresponds to 8× their vCPU count. We run the full TPC-H suite twice per network stack, both repetitions on a single spot instance. We further pin the experiment to a single S3 endpoint to minimize noise introduced by the object store. Consistent with the results of Durner et al. [10], we found that S3 cache pre-warming does not significantly affect performance and therefore avoid it.

Figure 4 shows the result. Mininet is faster on all 22 queries, by a geometric mean of 1.22× and 1.18× on the summed wall time, ranging from 1.71× on Q11 down to a narrow 1.03× on Q01 and Q21. The differences in miniosv performance offset to Linux are mostly due to the different workloads of the queries. Our current prototype statically configures the number of reserved cores and maximum number of connections in flight. This static configuration fits better for some queries than for others. We see, however, that even with this static configuration across all queries, mininet provides at least competitive performance to its Linux competitor.

6 Future Work

This section discusses some shortcomings of the current design and prototype and suggests improvements.

Advanced Software Features. GRO and other more advanced software features can be implemented in the mininet stack to further improve performance in cases where no jumbo frames are available.

Accounting for object store intricacies. Similar to Anyblob, mininet can be further optimized to better deal with S3 behavior. This includes optimizing the number and sizes of requests and preferring S3 endpoints for new connections that have performed better in the past [10].

Per Request Scalability. One flow hashes to one queue, which is managed by a single core. This limits the bandwidth of a single flow to the per-core reachable bandwidth. While this may not be an issue for plain HTTP requests, where per-core bandwidth, as shown in Section 5, exceeds 35 Gbit/s, TLS significantly reduces it. For this edge case, the design could be extended to automatically split flows into smaller requests, which can be distributed between multiple cores.

Dynamic Configuration. When using the producer/consumer model, mininet currently statically allocates a number of networking cores. This can be refined by monitoring how much these workers are actually utilized and scaling them up or down accordingly. The current prototype doesn't implement this, as miniOSv doesn't

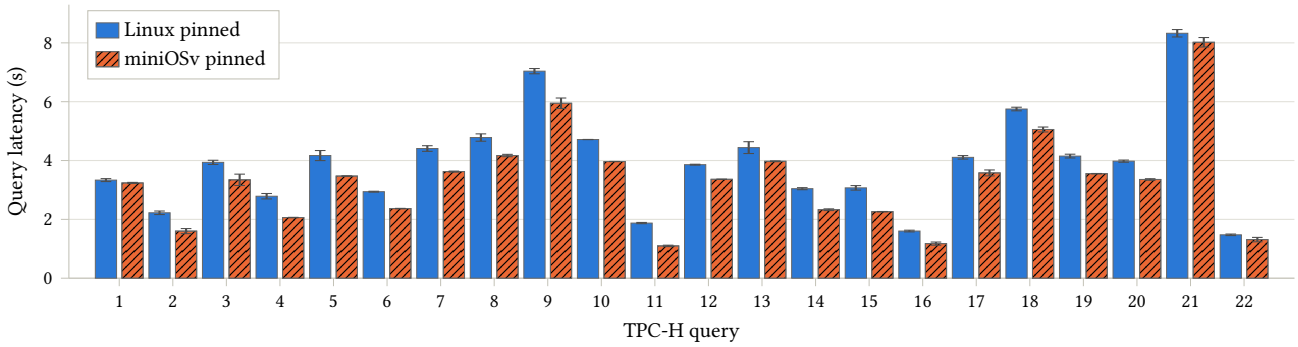


Figure 4: TPC-H at scale factor 100 on `r6in.8xlarge`, DuckDB over plain HTTP, 256 threads on both arms, both pinned to the same S3 endpoint. Bars are the mean of two runs, whiskers ±1 standard deviation.

provide a load balancer that could redistribute work once the number of usable cores to the application changes.

Full Application Integration. The producer/consumer model used in the current DuckDB integration is the fast and easy way to incorporate mininet into an analytical engine. Future work may explore using the low-level run-to-completion model to further increase control over the network stack and improve performance. Note that this modification also makes the dynamic configuration of polling cores in the kernel unnecessary, as the database now has full control over the configuration.

Zero-Copy. By leveraging the unmodified smoltcp and rustls crates, our implementation deliberately prioritizes low engineering effort over zero-copy. Further improving the reachable per-core bandwidth may require revisiting this decision and either using a modified version of the Rust libraries or implementing some functionality from scratch.

7 Conclusion

In this paper, we introduce mininet, a novel, purpose-built network stack design to provide high bandwidth for analytical query processing in the cloud. By leveraging the unikernel architecture, mininet overcomes the limitations of a traditional, kernel-native networking stack without introducing the complexity associated with kernel bypass mechanisms. Although our prototype leaves room for optimization, our evaluation shows that the current version achieves higher per-core bandwidth during bulk transfers and reduces the latency of every TPC-H query in DuckDB.

References

[1] Nikos Armenatzoglou, Sanuj Basu, Naga Bhanoori, Mengchu Cai, Naresh Chainani, Kiran Chinta, Venkatraman Govindaraju, Todd J. Green, Monish Gupta, Sebastian Hillig, Eric Hotinger, Yan Leshinksy, Jintian Liang, Michael McCreedy, Fabian Nagel, Ippokratis Pandis, Panos Parchas, Rahul Pathak, Orestis Polychroniou, Foyzur Rahman, Gaurav Saxena, Gokul Soundararajan, Sriram Subramanian, and Doug Terry. 2022. Amazon Redshift Re-invented. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 2205–2217. doi:10.1145/3514221.3526045

[2] Shinichi Awamoto and Michio Honda. 2025. Opening Up Kernel-Bypass TCP Stacks. In *2025 USENIX Annual Technical Conference (USENIX ATC '25)*. USENIX Association, Boston, MA, USA, 249–262. https://www.usenix.org/conference/atc25/presentation/awamoto

[3] AWS. 2026. Data Protection in Amazon EC2: Encryption in Transit. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/data-protection.html#encryption-transit. Accessed 2026-10-06.

[4] AWS. 2026. Introducing Amazon EC2 C8in and C8ib instances. https://aws.amazon.com/about-aws/whats-new/2026/04/amazon-ec2-c8in-c8ib-instances-ga/. Accessed 2026-10-06.

[5] Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. 2014. IX: A Protected Dataplane Operating System for High Throughput and Low Latency. In *11th USENIX Symposium on Operating Systems Design and Implementation* (Broomfield, CO, USA) *(OSDI '14)*. USENIX Association, Berkeley, CA, USA, 49–65. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/belay

[6] Fabian Bonk and Paul Emmerich. 2019. Networking in MirageOS. In *Proceedings of the Seminar Innovative Internet Technologies and Mobile Communications (IITM), WS 18/19*. Chair of Network Architectures and Services, Technical University of Munich, Munich, Germany, 47–51. doi:10.2313/NET-2019-06-1_10

[7] Alfred Bratterud, Alf-Andre Walla, Hårek Haugerud, Paal E. Engelstad, and Kyrre Begnum. 2015. IncludeOS: A Minimal, Resource Efficient Unikernel for Cloud Services. In *IEEE 7th International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, Vancouver, BC, Canada, 250–257. doi:10.1109/CloudCom.2015.89

[8] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *Proceedings of the 2016 International Conference on Management of Data* (San Francisco, California, USA) *(SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 215–226. doi:10.1145/2882903.2903741

[9] DPDK Project. 2021. Toeplitz Hash Library: Predictable RSS. https://doc.dpdk.org/guides/prog_guide/toeplitz_hash_lib.html. Accessed 2026-09-30.

[10] Dominik Durner, Viktor Leis, and Thomas Neumann. 2023. Exploiting Cloud Object Storage for High-Performance Analytics. *Proc. VLDB Endow.* 16, 11 (2023), 2769–2782. doi:10.14778/3611479.3611486

[11] Niklas Eiling, Martin Kröning, Jonathan Klimt, Philipp Fensch, Stefan Lankes, and Antonello Monti. 2023. GPU Acceleration in Unikernels Using Cricket GPU Virtualization. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*. ACM, Denver, CO, USA, 1588–1595. doi:10.1145/3624062.3624236

[12] Tom Herbert and Willem de Bruijn. 2024. Scaling in the Linux Networking Stack. Linux kernel documentation, https://docs.kernel.org/networking/scaling.html. Accessed 2026-09-27.

[13] Nur Hussein. 2017. IncludeOS: a unikernel for C++ applications. LWN.net. https://lwn.net/Articles/728682/

[14] EunYoung Jeong, Shinae Woo, Muhammad Jamshed, Haewon Jeong, Sunghwan Ihm, Dongsu Han, and KyoungSoo Park. 2014. mTCP: A Highly Scalable User-level TCP Stack for Multicore Systems. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI '14)*. USENIX Association, Seattle, WA, USA, 489–502. https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/jeong

[15] Antoine Kaufmann, Tim Stamler, Simon Peter, Naveen Kr. Sharma, Arvind Krishnamurthy, and Thomas Anderson. 2019. TAS: TCP Acceleration as an OS Service. In *Proceedings of the Fourteenth EuroSys Conference 2019 (EuroSys '19)*. ACM, Dresden, Germany, 1–16. doi:10.1145/3302424.3303985

[16] Avi Kivity, Dor Laor, Glauber Costa, Pekka Enberg, Nadav Har'El, Don Marti, and Vlad Zolotarov. 2014. OSv—Optimizing the Operating System for Virtual Machines. In *2014 USENIX Annual Technical Conference* (Philadelphia, PA, USA) *(USENIX ATC '14)*. USENIX Association, Berkeley, CA, USA, 61–72. https://www.usenix.org/conference/atc14/technical-sessions/presentation/kivity

[17] Simon Kuenzer, Vlad-Andrei Bădoiu, Hugo Lefeuvre, Sharan Santhanam, Alexander Jung, Gaulthier Gain, Cyril Soldani, Costin Lupu, Ştefan Teodorescu, Costi Răducanu, Cristian Banu, Laurent Mathy, Răzvan Deaconescu, Costin Raiciu, and Felipe Huici. 2021. Unikraft: Fast, Specialized Unikernels the Easy Way. In *Proceedings of the Sixteenth European Conference on Computer Systems (EuroSys '21)*. ACM, Online Event, United Kingdom, 376–394. doi:10.1145/3447786.3456248

[18] Hsuan-Chi Kuo, Dan Williams, Ricardo Koller, and Sibin Mohan. 2020. A Linux in Unikernel Clothing. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20)*. ACM, Heraklion, Greece, 1–15. doi:10.1145/3342195.3387526

[19] Stefan Lankes, Simon Pickartz, and Jens Breitbart. 2016. HermitCore: A Unikernel for Extreme Scale Computing. In *Proceedings of the 6th International Workshop on Runtime and Operating Systems for Supercomputers (ROSS '16)*. ACM, Kyoto, Japan, 1–8. doi:10.1145/2931088.2931093

[20] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft. 2013. Unikernels: Library Operating Systems for the Cloud. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems* (Houston, Texas, USA) *(ASPLOS '13)*. Association for Computing Machinery, New York, NY, USA, 461–472. doi:10.1145/2451116.2451167

[21] MiniOSv contributors. 2026. miniosv: A Radically Slimmed-Down Hard Fork of the OSv Unikernel. https://github.com/miniosv/miniosv. Accessed 2026-09-28.

[22] Lukas Neef. 2026. *High-Performance Networking in the Cloud.* Master's thesis. Technical University of Munich.

[23] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: An Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1981–1984. doi:10.1145/3299869.3320212

[24] Ali Raza, Thomas Unger, Matthew Boyd, Eric B. Munson, Parul Sohal, Ulrich Drepper, Richard Jones, Daniel Bristot de Oliveira, Larry Woodman, Renato Mancuso, Jonathan Appavoo, and Orran Krieger. 2023. Unikernel Linux (UKL). In *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23)*. ACM, Rome, Italy, 590–605. doi:10.1145/3552326.3587458

[25] Eric Rescorla. 2018. *The Transport Layer Security (TLS) Protocol Version 1.3.* RFC 8446. Internet Engineering Task Force. doi:10.17487/RFC8446

[26] Rustls contributors. 2026. Rustls: A modern TLS library in Rust. https://github.com/rustls/rustls. Accessed 2026-09-28.

[27] smoltcp contributors. 2026. smoltcp: a TCP/IP stack designed for bare-metal, real-time systems. https://github.com/smoltcp-rs/smoltcp. Accessed 2026-09-28.

[28] Tencent Cloud. 2017. F-Stack: A High-Performance User-Space Network Development Kit Based on DPDK. https://github.com/F-Stack/f-stack. Accessed 2026-10-06.

[29] The Linux Kernel Development Community. 2026. Linux Kernel Driver for Elastic Network Adapter (ENA) Family. Linux kernel documentation, https://docs.kernel.org/networking/device_drivers/ethernet/amazon/ena.html. Accessed 2026-09-28.

[30] Midhul Vuppalapati, Justin Miron, Rachit Agarwal, Dan Truong, Ashish Motivala, and Thierry Cruanes. 2020. Building An Elastic Query Engine on Disaggregated Storage. In *17th USENIX Symposium on Networked Systems Design and Implementation* (Santa Clara, CA, USA) *(NSDI '20)*. USENIX Association, Berkeley, CA, USA, 449–462. https://www.usenix.org/conference/nsdi20/presentation/vuppalapati

[31] Irene Zhang, Amanda Raybuck, Pratyush Patel, Kirk Olynyk, Jacob Nelson, Omar S. Navarro Leija, Ashlie Martinez, Jing Liu, Anna Kornfeld Simpson, Sujay Jayakar, Pedro Henrique Penna, Max Demoulin, Piali Choudhury, and Anirudh Badam. 2021. The Demikernel Datapath OS Architecture for Microsecond-scale Datacenter Systems. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. ACM, Virtual Event, Germany, 195–211. doi:10.1145/3477132.3483569

[32] Xinjing Zhou, Viktor Leis, Xiangyao Yu, and Michael Stonebraker. 2025. OLTP Through the Looking Glass 16 Years Later: Communication is the New Bottleneck. In *15th Conference on Innovative Data Systems Research, CIDR 2025, Amsterdam, The Netherlands, January 19-22, 2025*. www.cidrdb.org, Amsterdam, The Netherlands. https://vldb.org/cidrdb/2025/oltp-through-the-looking-glass-16-years-later-communication-is-the-new-bottleneck.html